MyCodeSchool

Code review — matrici

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-21 — sha 88765da

Am rulat fiecare punct din `temaExercitiiPDF.h` pe Test 1 și Test 2 din fișă (fișierele din `teste/`), apoi pe încă un set de date inventat ca să strice. Compilează curat.

Punct	Test 1 + Test 2 din fișă	Testul inventat
1a–1f, 2b, 2c, 2h, 3a, 3b, 3d, 3e, 3f	✅	✅
2e–2f	✅	🔴 iese din vector la un număr negativ (B2)
2g	✅	🔴 iese din vector la negativ (B2) · afișează cifra 0 (B5)
2a	✅	🔴 un maxim 0 sau o linie negativă dau `10` (B3, B4)
1g	✅	🔴 nu vede cifrele lui 0 și ale negativelor (B2, B3)
2d	✅	🔴 colțul negativ dă `-38` (B2)
2i–2j	✅	🟡 căutarea nu e eficientă (M2)
3c	🔴 **pică pe ambele teste** (B1)	

✅ 22 din 23 de puncte trec ambele teste din fișă. La 2e ai tratat separat cazul `aux == 0` (`f[0]++`): e exact regula care lipsește în B3. Ai scris-o deja o dată.

✅ `//??` pus pe 3c a fost instinctul bun: e singurul punct greșit. Pe 1d, 3e și 3f semnul e în plus, sunt corecte.

🔴 B1 — 3c: lipsește baza 16, iar un negativ trece drept superprim

`temaExercitiiPDF.h:563–584`

CUM E ACUM: Test 1 dă `233 233 7 3 23 3 29 29`, iar fișa cere `E9 E9 7 3 17 3 1D 1D`. Test 2 dă `-13 31`, iar fișa cere `1F`.

DE CE: sunt două greșeli. Prima: numerele găsite sunt cele bune, dar cerința e să le transformi în baza 16, iar transformarea lipsește. A doua: `while (n > 0)` nu intră deloc în buclă pentru `-13`, așa că funcția ajunge direct la `return true`. Un „nu am verificat nimic" iese ca „da". Același lucru pentru `0`.

FIX:

```
bool superprim(int n) {
    if (n <= 1) {
        return false;
    }
    while (n > 0) {
        if (prim(n) == false) {
            return false;
        }
        n = n / 10;
    }
    return true;
}
```

```
void afisareBaza16(int n) {
    char cifre[] = "0123456789ABCDEF";
    char rez[20];
    int k = 0;
    while (n != 0) {
        rez[k] = cifre[n % 16];
        k++;
        n = n / 16;
    }
    for (int i = k - 1; i >= 0; i--) {
        cout << rez[i];
    }
    cout << " ";
}
```

și în `ex3c` : `afisareBaza16(m[i][j]);` în loc de `cout << m[i][j] << " ";` .

🔴 B2 — Cifrele unui negativ: `f[-3]++` scrie în afara vectorului

`temaExercitiiPDF.h:374` (2e) · `:426` (2g)

CUM E ACUM: cu `-23` pe o linie impară și o coloană pară, programul scrie în memoria de dinaintea lui `f` . AddressSanitizer: `index -3 out of bounds for type 'int[10]'` . Același lucru la 2g, cu `-35` în interior.

DE CE: în C++, `%` păstrează semnul primului număr: `-23 % 10` este `-3` , nu `3` . Indicele `f[-3]` nu dă eroare, ci scrie în căsuța din memorie aflată înaintea lui `f` , care aparține altei variabile. Programul merge mai departe cu date stricate. Convenția din fișă spune: „cifrele unui număr negativ se iau fără semn”. La 3b și 3f ai aplicat-o cu `abs(...)` , aici nu.

FIX: la ambele puncte, `int aux = abs(m[i][j]);`

➕ Aceeași regulă e încălcată, fără crash, dar cu răspuns greșit, în: - `:201` (1g): `-111` are 3 cifre binare, dar tu numeri 0, pentru că `-1` nu e nici `0` , nici `1` . Cu `{10, 2}, {-111, 3}` iese `linia 0` , corect e `linia 1` . Fix: `int aux = abs(m[i][j]);` - `:338` (2d): `cif_control(-38)` întoarce `-38` , pentru că `-38 > 9` e fals. Corect e `2` . Fix: `n = abs(n);` pe prima linie din funcție.

🔴 B3 — Numărul 0 are o cifră, dar `while (n != 0)` nu i-o vede

`temaExercitiiPDF.h:227–238` (`cif_min`) · `:202` (1g)

CUM E ACUM: linia `{0, 0}` la 2a: maximul e 0, iar `cif_min(0)` întoarce `10`. Rezultatul e `107`, corect e `7`. La 1g, cu `{2, 3}, {0, 5}`, iese `linia 0`, deși singurul număr cu cifră binară este `0`, aflat pe linia 1.

DE CE: bucla `while (n != 0)` taie câte o cifră până rămâne 0. Pentru `n = 0`, condiția e falsă de la început, deci bucla nu rulează niciodată și numărul pare să nu aibă cifre. La `cif_min`, asta înseamnă că rămâne valoarea de start, `10`, care nu e cifră. La 2e și 2g ai prins cazul ăsta cu `if (aux == 0) f[0]++;`. Aici lipsește.

FIX: bucla `do … while` rulează măcar o dată, deci prinde și cifra lui 0:

```
int cif_min(int n) {
    n = abs(n);
    int min = 10;
    do {
        int cif = n % 10;
        if (cif < min) {
            min = cif;
        }
        n = n / 10;
    } while (n != 0);
    return min;
}
```

La 1g, aceeași schimbare: `do { … } while (aux != 0);`.

🔴 B4 — 2a: maximul pornește de la 0

`temaExercitiiPDF.h:245`

CUM E ACUM: linia `{-5, -7}` are maximul `-5`, deci cifra 5. La tine, maximul rămâne `0`, iar `cif_min(0)` dă `10`. Rezultatul e `107`, corect e `57`.

DE CE: un maxim pornit de la 0 funcționează doar dacă linia are măcar un număr pozitiv. Dacă toate sunt negative, niciunul nu trece de `> 0`, iar „maximul” rămâne valoarea de start, care nici nu e în matrice. Startul sigur e chiar primul element de pe linie.

FIX: `max = m[i][0];`

🔴 B5 — 2g: cifra 0 trebuie exclusă

`temaExercitiiPDF.h:434`

CUM E ACUM: cu `35` singur în interior iese `01246789`, iar cerința spune „se exclude și cifra 0". Corect e `1246789`. Ambele teste din fișă au un 0 în interior, de aceea n-ai văzut.

FIX: `for (int i = 1; i < 10; i++)`

🟡 M1 — Matricea nu se citește din `matrice.in`

`matrici.cpp:4-16`. Fișa cere citirea din fișier la toate cele 3 probleme. În `algoritmi.h` ai deja `citire(a, n, m)`, dar nu e folosită: matricea e scrisă de mână în `main()`, și doar pentru problema 3. Copiază `teste/p1-test1.in` peste `matrice.in` și rulează:

```
int a[100][100], n, m;
citire(a, n, m);
ex1a(a, n, m);
```

🟡 M2 — 2j: „căutați într-un mod eficient" = căutare binară

`temaExercitiiPDF.h:493-498`. Coloana e sortată la punctul i) tocmai ca să poți căuta binar. Tu o parcurgi de la cap la coadă. Ține la fiecare pas regula: intervalul `[st, dr]` trebuie să scadă.

```
int st = 0, dr = l - 1, poz = -1;
while (st <= dr && poz == -1) {
    int mij = (st + dr) / 2;
    if (m[mij][col] == k) {
        poz = mij;
    }
    else if (m[mij][col] < k) {
        st = mij + 1;
    }
    else {
        dr = mij - 1;
    }
}
```

🟡 M3 — `afisare` e scrisă de 3 ori

`algoritmi.h:39`, `parcurgerea_matricilor.h:7`, `temaExercitiiPDF.h:63`. Dacă incluzi `algoritmi.h` pentru `citire` (M1), compilarea pică cu `error: redefinition of 'afisare'`. Păstreaz-o doar în `algoritmi.h` și include `algoritmi.h` în `temaExercitiiPDF.h`.

🟢 Cleanups

- C1 `:531` `bool vf` nefolosit în `vfBinar`.
- C2 `:149` `long long v[10000]`: elementele sunt `int`, iar matricea are cel mult 50 × 100, deci ajunge `int v[5000]`.
- C3 `:174`, `:195` `-9999999` ca start pentru maxim: la fel ca la B4, pornește de la prima sumă sau de la primul element.

Before / After

	Acum	Corect
B1	`while (n > 0) { … }` `return true;`	`if (n <= 1) return false;` + afișare în baza 16
B2	`int aux = m[i][j];`	`int aux = abs(m[i][j]);`
B3	`while (n != 0) { … }`	`do { … } while (n != 0);`
B4	`max = 0;`	`max = m[i][0];`
B5	`for (int i = 0; …)`	`for (int i = 1; …)`

Întrebări

1. Cât face `-47 % 10` în C++? Ce indice primește atunci `f[cif]++`?
2. De câte ori se execută corpul lui `while (n != 0)` pentru `n = 0`? Dar al lui `do … while (n != 0)`?
3. Pe linia `{-8, -3, -6}`, ce iese ca maxim dacă pornești de la `max = 0`? Dar dacă pornești de la `max = m[i][0]`?

💡 Toate cele 5 🔴 trec pe Test 1 și Test 2. Testul care le prinde: **un negativ, un 0 și o linie numai cu negative**. Adaugă-l la fiecare punct care lucrează cu cifre.